Chapter 10 - Object Oriented Programming

Solving a problem by creating object is one of the most popular approach in programming. This is called - object - oriented programming.

This concept focuses on using reusable code (DRY Principle).

Class.

A class is a blueprint for creating object.

Combines info to create a valid application ← Blank Form → Filled by a Student

↓ ↓

Application of the student

Class ⟹ object instantiation ⇒ Object

contains info to create a valid object

Syntax :

Class Employee: # Class name is written in pascal ~~class~~ case.

Methods & Variables.

Object : An object is an instantiation of a class. When class is defined, a template (info) is defined. Memory is allocated only after object instantiation.

Object of a given class can invoke the methods available to it without revealing the implementation details to the user (Abstraction & Encapsulation).

Moodelling A Problem In OOP's

We identify the following in our program

- Noun → Class → Employee
- Adjective → Attributes → name, age, salary
- Verbs → Methods → getSalary (), IncrementS

Class Attributes :

An attribute that belongs to the class rather then a perticular object.

Code Example :

```
Class Employee :
    Company = 'Google'          # Specific to each class

Micro = Employee ()             # Object Instantation
Micro.Company                   # Prints 'Google'
Employee.Company = 'YouTube'    # changing
                                  class Attribute
```

Instance Attributes: An attribute that belongs to the instance (object).

→ Assume class from previous example:

```
Code:
  Micro.name   = 'Micro'
  Micro.Salary = '300K'  # Adding instance Att
```

Note: Instance attribute takes preference over class attribute during assignment & retrieval.

When looking up for micro.attribute it checks if:

1> attribute is present in object?
2> attribute is present in class?

Self Parameter:

Self refers to the instance of class. It is automatically passed with a function call from an object.

```
Micro.getSalary()   # here self is Micro

# equalent to Employee.getSalary(micro)
```

→ The function .getSalary() is defined as:

```
Code:
Class Employee:
    Company = 'Google'
    def getSalary(self):
        Print("Salary is not there")
```

Static Method :

Sometimes we need a function that doesnot use the self-parameter. We can define a static method like this:

Code:

```
@staticmethod          # decorator to mark greet as
def greet():             a static method.
    print ('Hello user')
```

__INIT__() Constructor :

- __init__() is a special method which is first run as soon as the object is created.

- __int__() method is also known as constructor.

- It takes 'self' argument and can also take further arguments.

For example :

```
Class Employee :
    def __init__(self, name):
        self. name = name
    def getSalary(self):
        ....
Micro = Employee ('Micro').
```